CSE 4303
Data Structures

Topic: Range Queries

(Prefix sum, Sparse Table)

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Prefix Sum

- *Prefix sum* helps to efficiently compute the ***sum of elements in a subarray***.

- Widely used in problems related to ***range sum queries***, cumulative frequency calculations, etc.

- Can answer *sum queries* in constant time $O(1)$, after an initial preprocessing step of $O(N)$.

pref[k] = sum of first k items

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

```
pref[0] = 0
pref[i] = ?      // general formula
```

pref[k] = sum of first k items

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

2

pref[0] = 0

pref[i] = pref[i-1] + a[i-1]

Find sum(L,R) // say (L,R)=(2,5)

idx	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Find sum(L,R) // say (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 of original array –
sum of items upto idx 2
= 14 – 5 = 9 // (1+3+1+4=9)

Find sum(L,R) // say (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 of original array –
sum of items upto idx 2

Sum(L,R) = ? // in general?

Find sum(L,R) // say (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 of original array –
sum of items upto idx 2

Sum(L,R) = pref[R+1] – pref[L]

Query: find sum(L,R)

Brute Force:

```
for each Q query [L,R]:
      sum=0
      for i=L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(?)$

Query: find sum(L,R)

Brute Force:

```
for each Q query [L,R]:
      sum=0
      for i=L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(?)$

Query: find sum(L,R)

Prefix sum:

```
for i=1…N          // preprocessing
      pref[i] = pref[i-1] + a[i-1]

for every query [L,R]:
      print (pref[R+1]-pref[L])
```

Complexity: $O(?)$

Query: find sum(L,R)

Brute Force:

```
for each Q query [L,R]:
      sum=0
      for i=L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(Q.N)$

Query: find sum(L,R)

Prefix sum:

```
for i=1…N          // preprocessing
      pref[i] = pref[i-1] + a[i-1]

for every query [L,R]:
      print (pref[R+1]-pref[L])
```

Complexity: $O(N+Q)$

Given a string S of length N

Answer Q queries: given L and R, find the most common character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

Given a string S of length N

Answer Q queries: given L and R, find the most common character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

```
a occurs 3 times
b - 2
c – 2

Ans: ‘a’
```

Given a string S of length N

Answer Q queries: given L and R, find the most common character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

Can we apply Prefix sum concept to improve the situation?

Given a string S of length N

Answer Q queries: given L and R, find the most common character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

Can we apply Prefix sum concept to improve the situation?

Can store freq of each letter upto each idx

Simply apply prefix sum formula!

Given a string S of length N

Answer Q queries: given L and R, find the most common character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

Useful idea while working with lots of Queries.
Just precompute once!

Can we apply Prefix sum concept to improve the situation?

Can store freq of each letter upto each idx

Simply apply prefix sum formula!



Prefix sum doesn't work with min/max

idx	0	1	2	3	4	5	6	7
a[i]	20	7	6	7	4	30	2	
pref[i]	0	20	20	20	20	20	30	30

Find max of Range (2,4)

- When to use prefix sum?
  - Range sum
  - XOR, products
  - Frequency counts
  - Doesn’t support update operation



Sparse Table

Range Minimum Queries in $O(1)$

Sparse Table

- Sparse Table is used to efficiently answer range queries in an **immutable** array.

- Provides fast queries (O(1) time complexity) after an O(N log N) preprocessing step.

- Stores ***range having length of power of 2*** instead of every possible range (hence the term “Sparse”)

Sparse Table

- **Best Use Case**: When the array is static (i.e., it does not change after initialization).

- **Ideal for**: Range Minimum Query (RMQ), Range Maximum Query (RMQ), and GCD/LCA queries.

- **Not suitable for**: Problems that require frequent updates (use Segment Tree instead).

Key Features

- Preprocessing Time: $O(\mathrm{N}\log N)$
- Query Time: $O(1)$ for idempotent operations (min, max, gcd, etc.)
- Space Complexity: $O(\mathrm{N}\log N)$

Concept

- A Sparse Table stores precomputed answers for overlapping ***ranges of powers of two***.

- For an array of size N, we define $table[i][j]$
  - where, i represents the starting index of the range.
  - j represents length of range as a power of 2 (i.e. 2^j) .

Concept

- A Sparse Table stores precomputed answers for overlapping ***ranges of powers of two***.

- For an array of size N, we define $table[i][j]$
  - where, i represents the starting index of the range.
  - j represents length of range as a power of 2 (i.e. 2^j) .

  - $table[3][2]$ represents range $(3, 3 + 2^2 - 1) = (3,6)$

Stores intervals of width $w = 2^j$

$(1,1)$
$(1,2)$
~~$(1,3)$~~
$(1,4)$
~~$(1,5)$~~
~~$(1,6)$~~
~~$(1,7)$~~
$(1,8)$

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	~~(2,4)~~
(1,4)	(2,5)
~~(1,5)~~	~~(2,6)~~
~~(1,6)~~	~~(2,7)~~
~~(1,7)~~	~~(2,8)~~
(1,8)	

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	~~(2,4)~~
(1,4)	(2,5)
~~(1,5)~~	~~(2,6)~~
~~(1,6)~~	~~(2,7)~~
~~(1,7)~~	~~(2,8)~~
(1,8)	

How is this represented in sparse table?

`table[i][j]=(i, i+`2^j`-1)`

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	~~(2,4)~~
(1,4)	(2,5)
~~(1,5)~~	~~(2,6)~~
~~(1,6)~~	~~(2,7)~~
~~(1,7)~~	~~(2,8)~~
(1,8)	

How is this represented in sparse table?

table[i][j]=(i, i+2^j-1)

So,

table[2][0] represents range [2,2]

table[2][1] represents range [2,3]

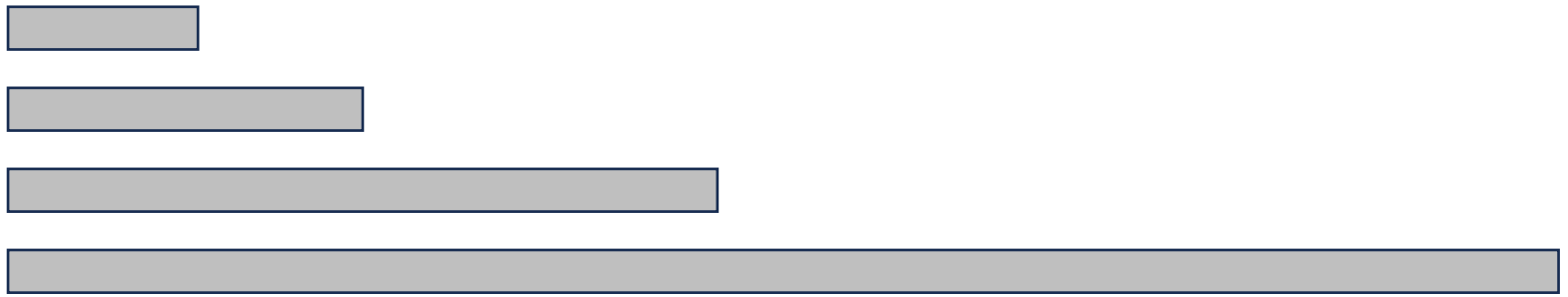
table[2][2] represents range [2,5]

table[2][3] represents range [2,9]

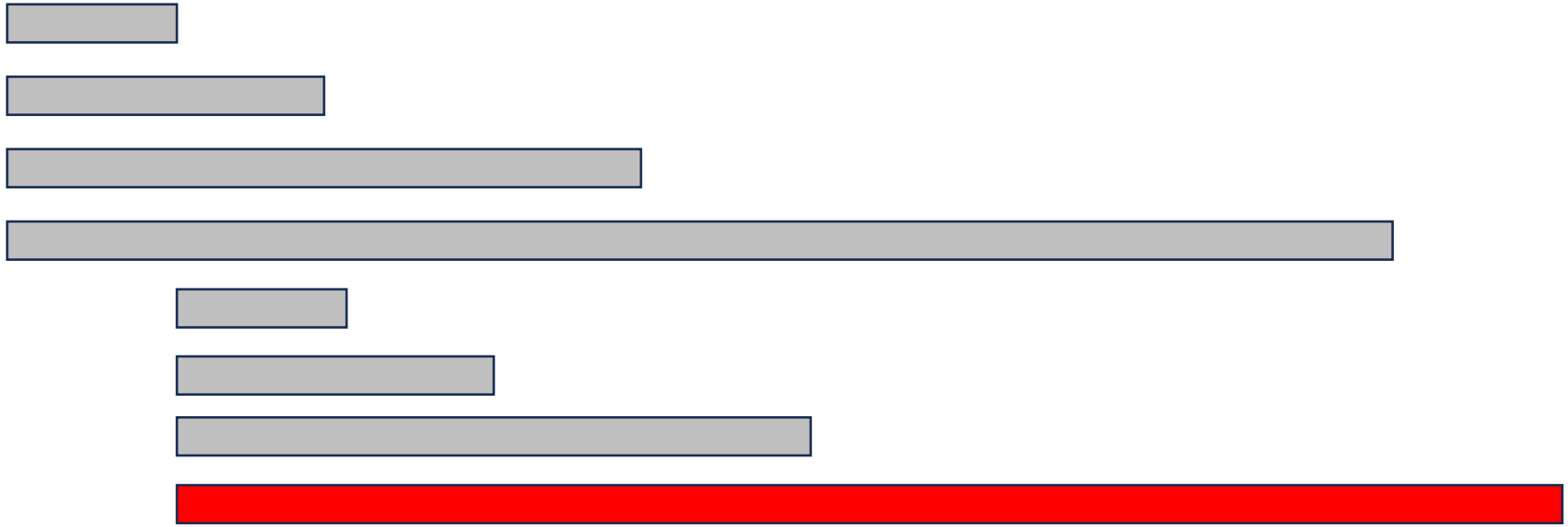
...

This expands upto limit N (#items)

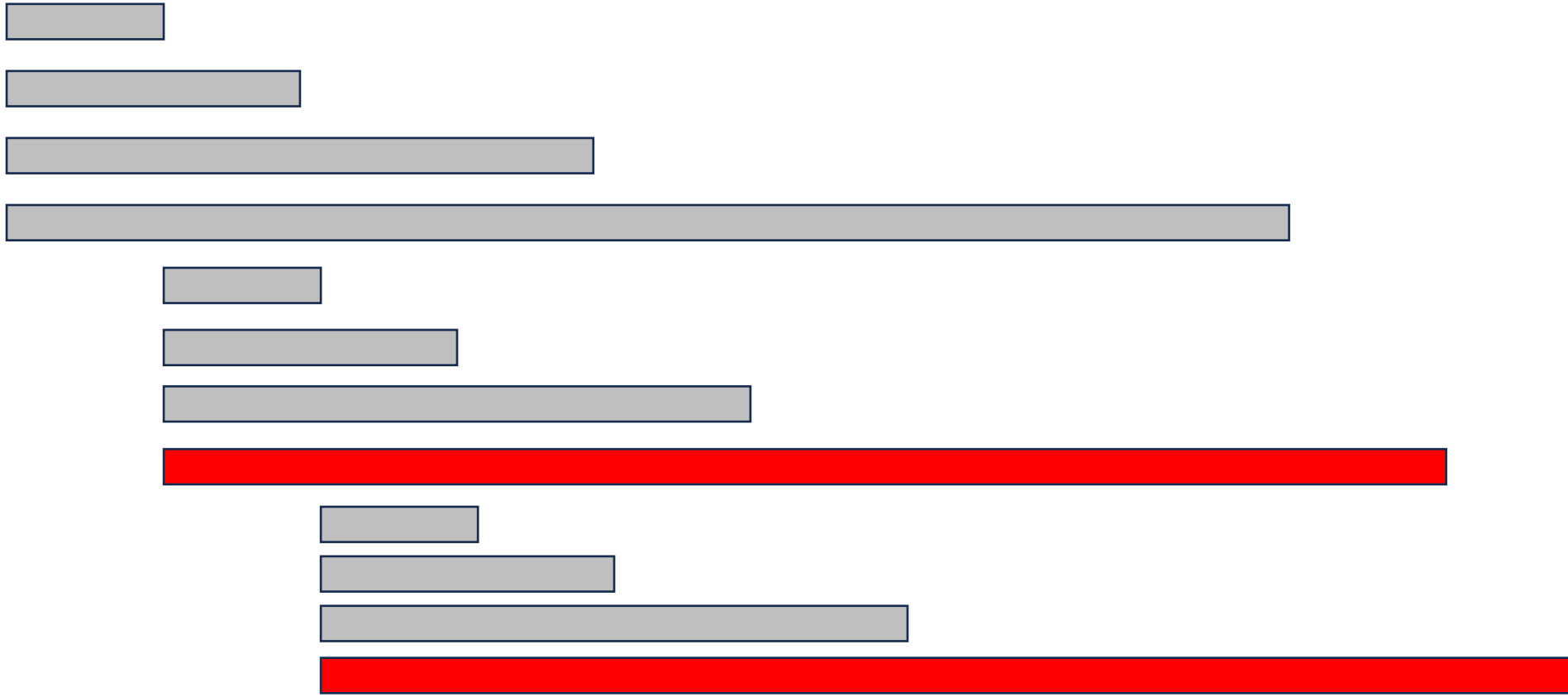
idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3



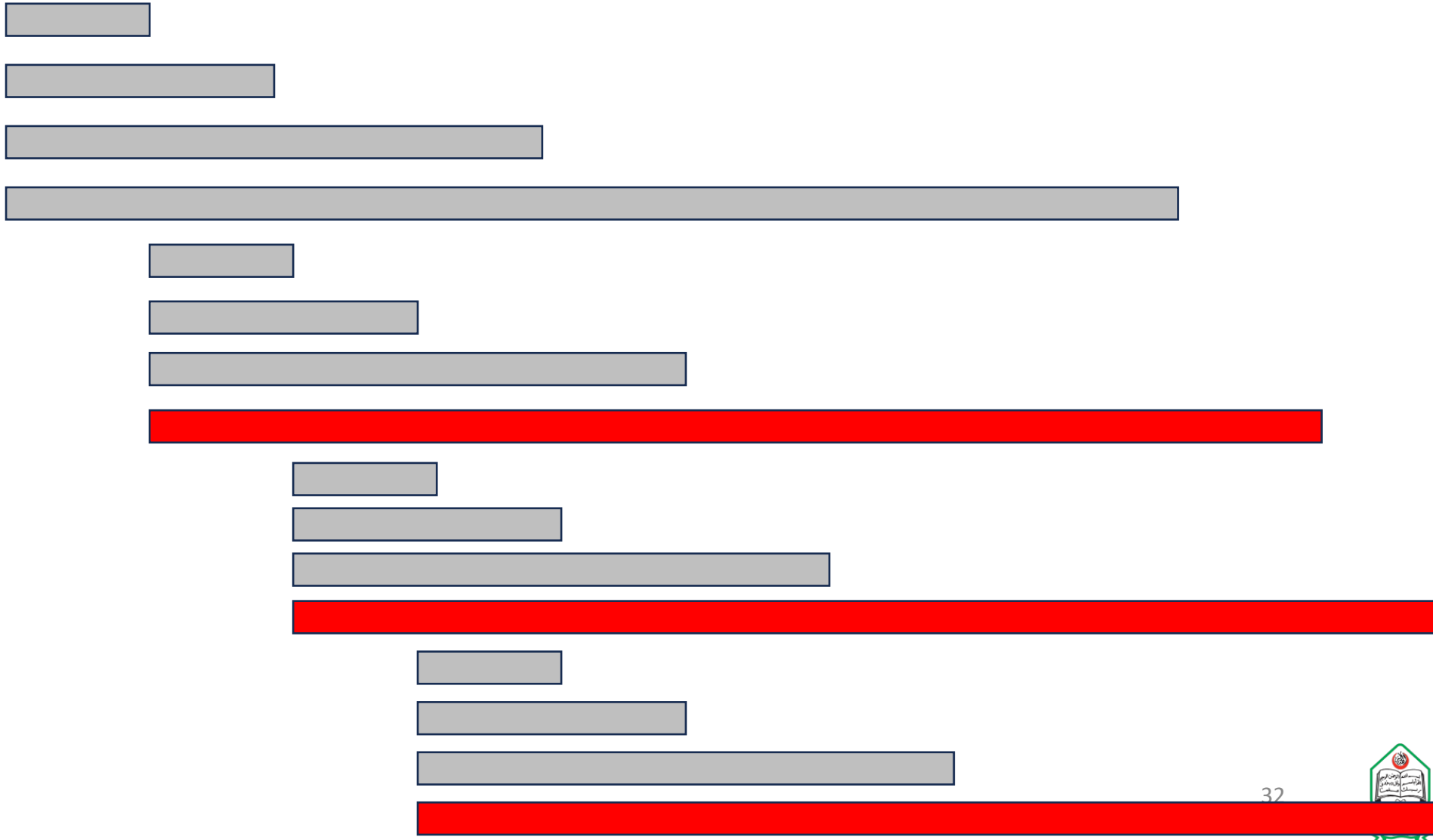
idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3



idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3



idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3



idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0				
1				
2				
3				
4				
5				
6				
7				

An Empty Sparse table

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	?			
1	?			
2	?			
3	?			
4	?			
5	?			
6	?			
7	?			

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$			
1	(1,1)			
2	(2,2)			
3	(3,3)			
4	(4,4)			
5	(5,5)			
6	(6,6)			
7	(7,7)			

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	?		
1	(1,1)	?		
2	(2,2)	?		
3	(3,3)	?		
4	(4,4)	?		
5	(5,5)	?		
6	(6,6)	?		
7	(7,7)	?		

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$		
1	(1,1)	(1,2)		
2	(2,2)	(2,3)		
3	(3,3)	(3,4)		
4	(4,4)	(4,5)		
5	(5,5)	(5,6)		
6	(6,6)	(6,7)		
7	(7,7)	~~(7,8)~~ Out of bound		

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$	$(0,0+2^2-1)$ $=(0,3)$	
1	(1,1)	(1,2)	(1,4)	
2	(2,2)	(2,3)	(2,5)	
3	(3,3)	(3,4)	(3,6)	
4	(4,4)	(4,5)	(4,7)	
5	(5,5)	(5,6)	~~(5,8)~~	
6	(6,6)	(6,7)	~~(6,9)~~	
7	(7,7)	~~(7,8)~~ Out of bound	~~(7,10)~~	

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$	$(0,0+2^2-1)$ $=(0,3)$	$(0,0+2^3-1)$ $=(0,7)$
1	(1,1)	(1,2)	(1,4)	(1,8)
2	(2,2)	(2,3)	(2,5)	~~(2,9)~~
3	(3,3)	(3,4)	(3,6)	~~(3,10)~~
4	(4,4)	(4,5)	(4,7)	~~(4,11)~~
5	(5,5)	(5,6)	~~(5,8)~~	~~(5,12)~~
6	(6,6)	(6,7)	~~(6,9)~~	~~(6,13)~~
7	(7,7)	~~(7,8)~~ Out of bound	~~(7,10)~~	~~(7,14)~~

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1) = (0,0)$	$(0,0+2^1-1) = (0,1)$	$(0,0+2^2-1) = (0,3)$	$(0,0+2^3-1) = (0,7)$
1	(1,1)	(1,2)	(1,4)	(1,8)
2	(2,2)	(2,3)	(2,5)	
3	(3,3)	(3,4)	(3,6)	
4	(4,4)	(4,5)	(4,7)	
5	(5,5)	(5,6)		
6	(6,6)	(6,7)		
7	(7,7)			

Sparse table will save info of these ranges only

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2			
1	3			
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

Lets fill the exact values

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

table[3][1] represents
$\min(3, 3 + 2^1 - 1)$
$= (3,4)$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

table[2][2] represents
$\min(2, 2 + 2^2 - 1)$
$= (2,5)$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

How to build this table efficiently?

Cant just check minimum of every range using loop!

Slow!

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

We must utilize this precomputed information!

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

Biggest precomputed range starting from 5 is $table[5][4] = [5,20]$

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

Now, what is the biggest precomputed range we can use?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

Now, what is the biggest precomputed range we can use:
$table[21][2]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

Finally use table[25][1]

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find min(5,26)

Query width = 26-5+1 = 22
This information is not directly stored in the table

How to use precomputed ranges to derive this?

$21 + 2^2 - 1$
$21, 24$

$5 + 2^0 - 1$
$= 5$

$5 + 2^1 - 1$
$5, 6$

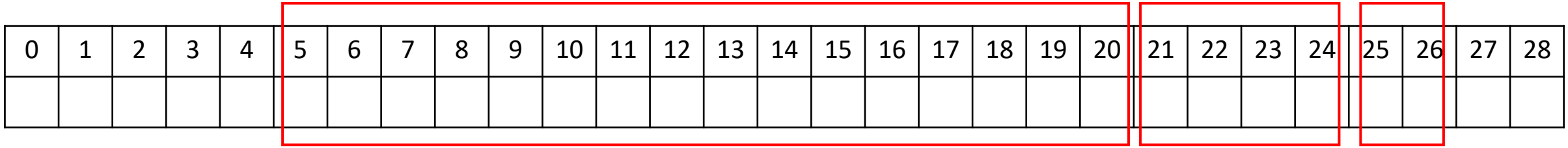
$5 + 2^2 - 1$
$5, 8$

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

Finally use $table[25][1]$



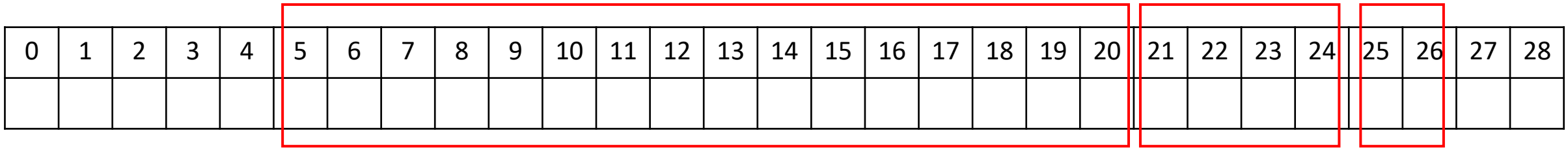
$$\min(5,26) = \min(\, table[5][4], [21][2], [25][1])$$

This is possible for any range of width w, as

every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of $16, 4, 2$ to find the total range



$$\min(5,26) = \min(table[5][4], [21][2], [25][1])$$

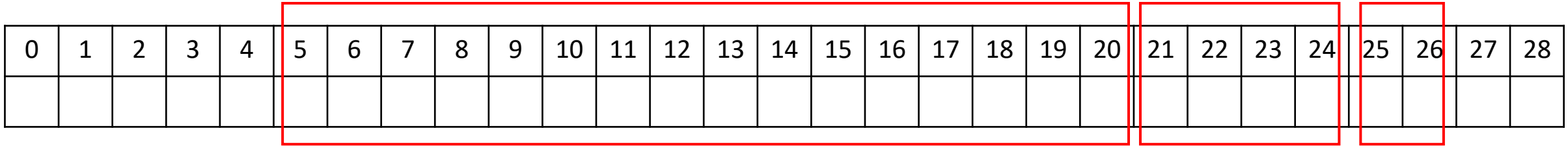
This is possible for any range of width w, as

every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of $16, 4, 2$ to find the total range

Complexity of answering range minimum query = $O(??)$



$$\min(5,26) = \min(\, table[5][4], [21][2], [25][1])$$

This is possible for any range of width w, as

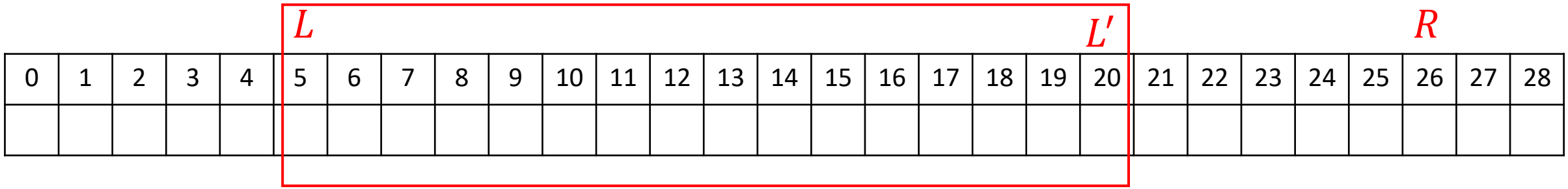
every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of $16, 4, 2$ to find the total range

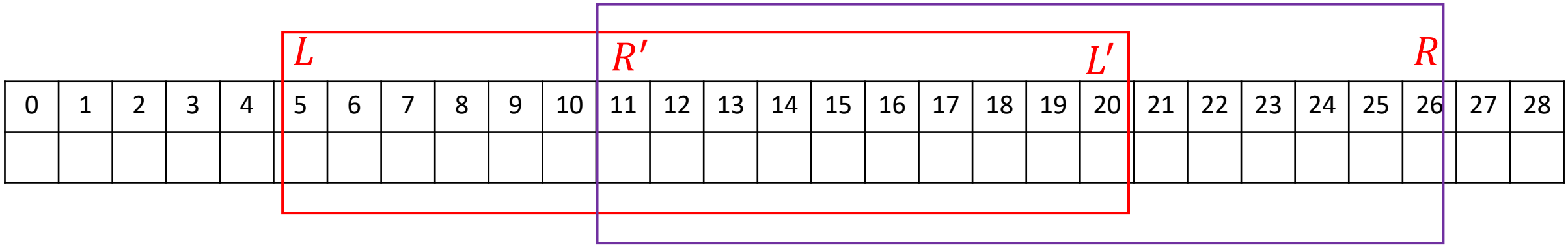
Complexity of answering range minimum query = $\boldsymbol{O(\log(w))}$

Can we do it in O(1)?



Find min(L,R) = min(5,26)
Window size=22

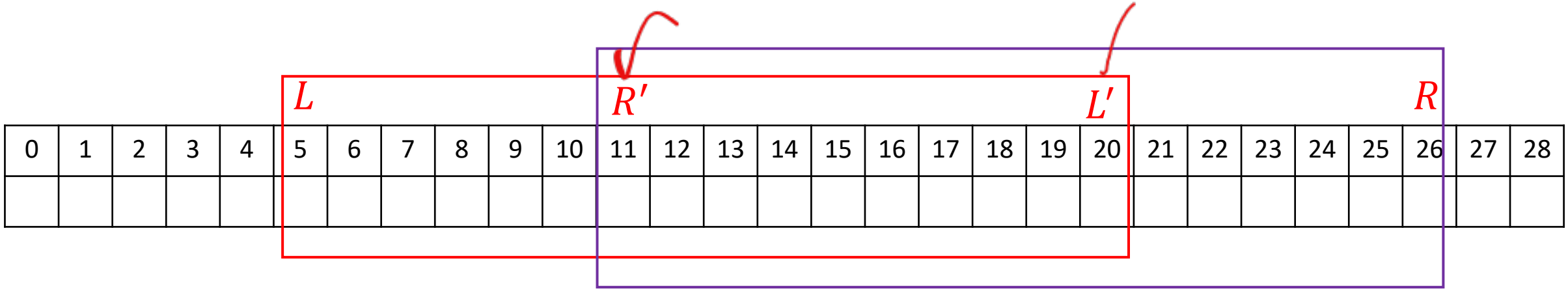
Biggest power of 2 less than 22 is 16



Find min(L,R) = min(5,26)
Window size=22

Biggest power of 2 less than 22 is 16

What if we use a similar size window starting from 16 blocks left to R?

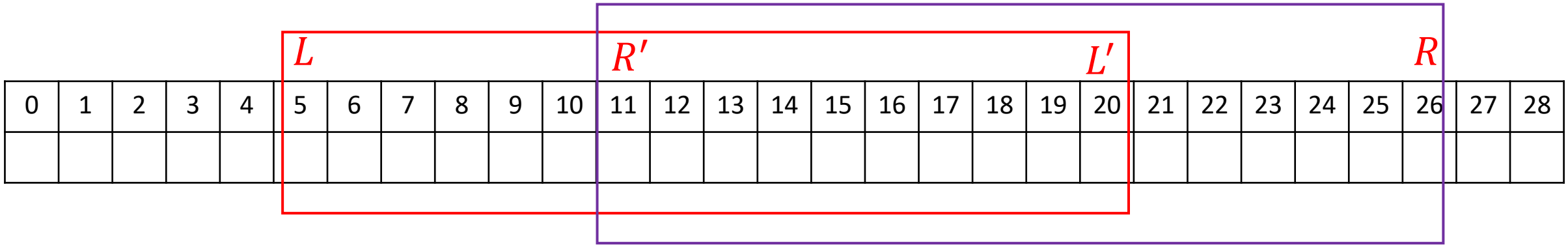


Find min(L,R) = min(5,26)
Window size=22

Biggest power of 2 less than 22 is 16

What if we use a similar size window starting from 16 blocks left to R?

$$\textit{Now, } \min(L,R) = \min(\min(L,L'),\min(R',R))$$
$$= \min(table[L][j], table[R-2^j][j]) \; ; where \; j = \log(windowSize)$$

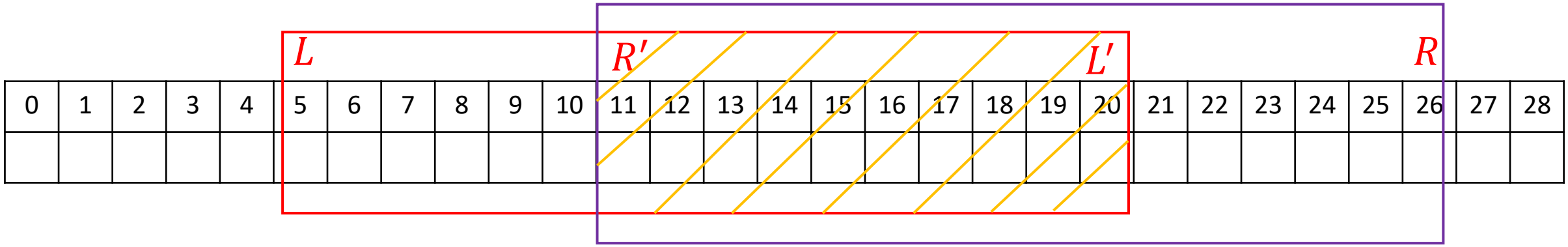


Find min(L,R) = min(5,26)
Window size=22

Biggest power of 2 less than 22 is 16

What if we use a similar size window starting from 16 blocks left to R?

$$\textit{Now, } \min(L,R) = \min(\min(L,L'),\min(R',R))$$
$$= \min(table[L][j], table[R-2^j][j]) \,;\, where\ j = \log(windowSize)$$
$$= \min(table[L][j], table[R-1 \ll j][j])$$



$$\min(L, R) = \min(\min(L, L'), \min(R', R))$$
$$= \min(table[L][j], table[R - 2^j][j]) \; ; where \; j = \log(windowSize)$$

The overlapped region doesn't cause any harm as it doesn't change the minimum value!

Time complexity for range minimum query is now $\boldsymbol{O(1)}$

```
int main() {
	// 1) read input
	int n; cin >> n;
	for(int i = 0; i < n; i++) {
		cin >> a[i];
		table[i][0] = a[i];
	}
```



```
}
```




```
int main() {
	// 1) read input
	int n; cin >> n;
	for(int i = 0; i < n; i++) {
		cin >> a[i];
		table[i][0] = a[i];
	}
	// 2) preprocessing, O(N*log(N))
	for(int j = 1; j < log(n); k++) {
		for(int i = 0; i + (1 << j) - 1 < n; i++) {
			table[i][j] = min(table[i][j-1],
					table[i+(1<<(j-1))][j-1]);
		}
	}
}
```

```
int main() {
	// 1) read input
	int n; cin >> n;
	for(int i = 0; i < n; i++) {
		cin >> a[i];
		table[i][0] = a[i];
	}
	// 2) preprocessing, O(N*log(N))
	for(int j = 1; j < log(n); k++) {
		for(int i = 0; i + (1 << j) - 1 < n; i++) {
			table[i][j] = min(table[i][j-1],
					table[i+(1<<(j-1))][j-1]);
		}
	}
	// 3) answer queries
	int q; cin >> q;
	for(int i = 0; i < q; i++) {
		int L, R; cin >> L >> R;
		cout << query(L, R) << "\n";
	}
}
```

```
int main() {
        // 1) read input
        int n; cin >> n;
        for(int i = 0; i < n; i++) {
                cin >> a[i];
                table[i][0] = a[i];
        }
        // 2) preprocessing, O(N*log(N))
        for(int j = 1; j < log(n); k++) {
                for(int i = 0; i + (1 << j) - 1 < n; i++) {
                        table[i][j] = min(table[i][j-1],
                                          table[i+(1<<(j-1))][j-1]);
                }
        }
        // 3) answer queries
        int q; cin >> q;
        for(int i = 0; i < q; i++) {
                int L, R; cin >> L >> R;
                cout << query(L, R) << "\n";
        }
}
```

```
int query(int L, int R) { // O(1)
        int window = R - L + 1;
        int j = log(window);
        return min(
                table[L][j],
                table[R-(1<<j)+1][j]);
}
```

References

- Range Query Basics: https://www.youtube.com/watch?v=8wJzUFZOqK4&ab_channel=KartikArora
- Prefix arrays: https://www.youtube.com/watch?v=-_Jj4U5V4N0&t=2s&ab_channel=KartikArora
- RMQ Sparse Table: https://www.youtube.com/watch?v=iaRvydtqLV4&ab_channel=KartikArora

Thank you